CO224 Computer Architecture
Lab 4.5 - Extended ISA Comprehensive Report

Group 02
E/22/151 E/22/056

June 18, 2026

1 Introduction

This document provides a technical breakdown of the ISA extensions implemented in the 8-bit baseline processor. The extensions include multiplication (`mult`), logical and arithmetic shifts (`sll`, `srl`, `sra`), rotate (`ror`), and branch-not-equal (`bne`). The report details instruction encoding, control unit modifications, structural hardware implementations strictly avoiding primitive operators, and critical path timing analysis.

2 Instruction Set Architecture (ISA) Extensions

The baseline ISA was extended utilizing unassigned opcodes starting from `0x08`. The encoding formats structurally mirror the baseline R-type and I-type instructions, minimizing additional decoding overhead in the Control Unit.

Table 1: Opcode Assignment and 32-bit Instruction Encoding

Operation	**Mnemonic**	**Opcode**	**Binary Encoding (`Opcode | Dest | Src1 | Src2/Imm`)**
Multiplication	`mult`	`0x08`	`00001000 | dddddddd | iiiiiiii | jjjjjjjj`
Shift Left Logical	`sll`	`0x09`	`00001001 | dddddddd | iiiiiiii | vvvvvvvv`
Shift Right Logical	`srl`	`0x0A`	`00001010 | dddddddd | iiiiiiii | vvvvvvvv`
Shift Right Arith.	`sra`	`0x0B`	`00001011 | dddddddd | iiiiiiii | vvvvvvvv`
Rotate Right	`ror`	`0x0C`	`00001100 | dddddddd | iiiiiiii | vvvvvvvv`
Branch Not Equal	`bne`	`0x0D`	`00001101 | vvvvvvvv | iiiiiiii | jjjjjjjj`

3 Control Unit Implementation

To respect the maximum limit of 8 distinct `ALUOP` states (dictated by the 3-bit signal width), functional unit sharing was enforced. Opcodes `0x09` through `0x0C` share `ALUOP = 101` (SHIFT), while a secondary 2-bit signal (`SHIFT_SEL`) resolves the specific shift variant. The `mult` instruction uses `ALUOP = 110`. The `bne` instruction utilizes the existing `SUB` operation (`ALUOP = 010`) combined with a unique `BNE_FLAG`.

```
always @(OPCODE) begin
    // Default initializations
    ALUOP = 3'b000; SHIFT_SEL = 2'b00;
```

```
    BEQ_FLAG = 1'b0; BNE_FLAG = 1'b0; WRITE_EN = 1'b1;

    case (OPCODE)
        8'h08: ALUOP = 3'b110;                    // mult
        8'h09: begin ALUOP = 3'b101; SHIFT_SEL = 2'b00; end // sll
        8'h0A: begin ALUOP = 3'b101; SHIFT_SEL = 2'b01; end // srl
        8'h0B: begin ALUOP = 3'b101; SHIFT_SEL = 2'b10; end // sra
        8'h0C: begin ALUOP = 3'b101; SHIFT_SEL = 2'b11; end // ror
        8'h0D: begin                              // bne
            ALUOP = 3'b010; // Reuse SUB hardware
            BNE_FLAG = 1'b1;
            WRITE_EN = 1'b0; // Branches do not write to registers
        end
    endcase
end
```

Listing 1: Control Unit Decoding Logic Extract

4 Structural Functional Units

Per laboratory constraints, behavioral shift (<<, >>) and multiplication ($*$) operators were strictly prohibited. The hardware was synthesized structurally to ensure deterministic gate-level implementation and adherence to the project requirements.

4.1 Unified Barrel Shifter (`shift_unit.v`)

To avoid behavioral shift operators, a structural multiplexer-based barrel shifter was implemented. A barrel shifter routes data through $\log_2(N)$ stages of 2-to-1 multiplexers. For an 8-bit operand, this requires exactly 3 stages. The control signals for these multiplexers are directly tied to the shift amount's 3 least significant bits (`AMT[2:0]`).

The shifter consolidates all four operations (`sll`, `srl`, `sra`, `ror`) into parallel logic paths within each stage. The final output is selected via a 4-to-1 multiplexer driven by the `SHIFT_SEL` signal.

```
// Layer 0: Shift by 1 bit conditionally based on AMT[0]
// SLL: Inject 1'b0 at LSB
wire [7:0] stg0_sll = AMT[0] ? {DATA[6:0], 1'b0} : DATA;
// SRL: Inject 1'b0 at MSB
wire [7:0] stg0_srl = AMT[0] ? {1'b0, DATA[7:1]} : DATA;
// SRA: Duplicate MSB (DATA[7]) for sign extension
wire [7:0] stg0_sra = AMT[0] ? {DATA[7], DATA[7:1]} : DATA;
// ROR: Wrap DATA[0] to the MSB position
wire [7:0] stg0_ror = AMT[0] ? {DATA[0], DATA[7:1]} : DATA;

// Layer 1: Shift by 2 bits conditionally based on AMT[1]
// Operating on the outputs of Layer 0
wire [7:0] stg1_sll = AMT[1] ? {stg0_sll[5:0], 2'b00} : stg0_sll;
wire [7:0] stg1_srl = AMT[1] ? {2'b00, stg0_srl[7:2]} : stg0_srl;
wire [7:0] stg1_sra = AMT[1] ? {{2{stg0_sra[7]}}, stg0_sra[7:2]} : stg0_sra;
wire [7:0] stg1_ror = AMT[1] ? {stg0_ror[1:0], stg0_ror[7:2]} : stg0_ror;
```

Listing 2: Barrel Shifter Stage Logic (Shift by 1 and 2)

This design decision ensures the maximum combinational delay is strictly limited to 3 cascaded multiplexer layers plus the final output selection multiplexer. It completely circumvents primitive operators while guaranteeing execution well within the 8-time-unit cycle constraint.

4.2 Combinational Array Multiplier (`mult_unit.v`)

Utilizing the * operator delegates multiplication to the synthesis tool, violating the lab constraint against simple operations. To resolve this, an 8x8 combinational array multiplier was constructed.

The multiplication process is split into partial product generation and accumulation. Partial products are generated using 64 parallel AND gates (implemented via structural masking). These products are then shifted structurally and accumulated using a linear cascade of 8-bit adders.

```
// Partial Products Masking (Logical AND mapping)
wire [7:0] pp0 = B[0] ? A : 8'b0;
wire [7:0] pp1 = B[1] ? {A[6:0], 1'b0} : 8'b0;
wire [7:0] pp2 = B[2] ? {A[5:0], 2'b00} : 8'b0;
// ... extends to pp7

// Structural accumulation via dedicated adder modules
wire [7:0] sum0, sum1, sum2, sum3, sum4, sum5, sum6;

// Instantiating basic structural adders
adder_8bit add0 (.A(pp0), .B(pp1), .OUT(sum0));
adder_8bit add1 (.A(sum0), .B(pp2), .OUT(sum1));
// ... continues down the cascade
adder_8bit add6 (.A(sum5), .B(pp7), .OUT(sum6));

// The output is truncated to 8 bits to fit the datapath width
assign PROD = sum6;
```

Listing 3: Combinational Multiplier Partial Product Accumulation

By enforcing the structural instantiation of adders, the exact gate-level propagation path is known. The 8-bit truncation is a deliberate design decision adhering to the 8-bit processor datapath constraints, meaning the upper 8 bits of the theoretical 16-bit product are discarded.

4.3 Branch Not Equal (`bne`) Evaluation

The `bne` logical evaluation is decoupled from the ALU's primary output path and processed natively within the PC Unit based on the ALU's `ZERO` flag.

The design decision to reuse the existing `SUB` operation for `bne` minimizes hardware overhead. The ALU computes `A - B`. If `A` and `B` are unequal, the result is non-zero, and the ALU `ZERO` flag evaluates to `0`.

```
// Evaluate branch condition
// BEQ requires ZERO flag to be high. BNE requires ZERO flag to be low.
wire is_beq_taken = BEQ_FLAG & ZERO;
wire is_bne_taken = BNE_FLAG & ~ZERO;
wire take_branch = is_beq_taken | is_bne_taken;

// PC Update Logic
always @(posedge CLK) begin
    if (RESET)
        PC <= 0;
    else if (take_branch)
        PC <= PC + 4 + (TARGET_OFFSET << 2); // Branch resolution
    else
        PC <= PC + 4; // Sequential execution
end
```

Listing 4: PC Unit Branch Resolution

This approach reuses existing datapaths maximally, preventing the need for a dedicated hardware comparator unit, thereby conserving area and routing complexity.

5 Critical Path and Timing Analysis

The baseline clock cycle is 8 time units. Modifying the ALU introduces additional propagation delays, altering the critical path.

- **Baseline Critical Path:** `PC` $\rightarrow$ `I-Cache` $\rightarrow$ `RegRead` $\rightarrow$ `ALU(ADD)` $\rightarrow$ `RegWrite`.
- **Shift Unit Latency:** The barrel shifter introduces 4 multiplexer layers (3 stages + `SHIFT_SEL` MUX). Assuming a 2-to-1 MUX delay of 1 time unit, the shift path delay is 4 units, safely within the bounds of standard ALU operations.
- **Multiplier Latency:** The array multiplier is the most significant bottleneck. 7 cascaded 8-bit ripple-carry adders produce $O(N \times M)$ gate delays. If propagation exceeds 8 time units, synchronous execution fails. In hardware simulation, replacing sequential ripple-carry adders with Carry-Lookahead Adders (CLAs) within the `mult_unit` restricts the maximum combinational delay, ensuring compliance with the single-cycle 8-time-unit constraint.

6 Conclusion

The ISA was successfully extended by systematically utilizing functional unit sharing via the 3-bit `ALUOP` signal. Dedicated, structurally synthesized Verilog modules for shifting and multiplication satisfied all algorithmic and primitive-operator constraints. The `bne` extension reused subtraction logic, validating the extensibility and modularity of the baseline 8-bit processor datapath.